

Heterogeneous Cognitive Transformers: Computational and Epistemic Coprocessors

Working Draft

August 2026

Abstract

We propose a blue-sky architecture in which a Transformer remains the general learned semantic substrate but is tightly coupled to specialized cognitive coprocessors. Two interleaved tracks are studied: *computational coprocessors*, which provide exact or structured operations such as arithmetic, logic, constraints, graph reasoning, algebra and code execution; and *epistemic coprocessors*, which detect emerging information needs and retrieve evidence from databases, vector stores, document indices, knowledge graphs and the web. Both can be activated during autoregressive inference through progressively richer semantic interrupts, from strict syntax and lexical heuristics to learned routing. Outputs become typed cognitive micro-state and may later be selectively materialized through PRA or native interfaces. Conventional tool calling and upfront RAG remain fallback/complementary mechanisms.

1 Motivation

Current LLM systems often collapse two different failures into “tool use.” A *competence deficit* occurs when the model has enough information but benefits from exact computation, e.g. multiplication or logical closure. A *knowledge deficit* occurs when the model lacks reliable, fresh, or source-specific evidence. The proposed runtime treats these as distinct cognitive needs.

2 Twin Tracks

2.1 Computational coprocessors

A computational interrupt means: “this emerging structure can be computed, inferred, checked, or transformed more reliably by a specialized engine.” Examples include calculators, Horn/Datalog engines, theorem provers, SMT solvers, graph engines, CAS systems, and code/simulation runtimes.

2.2 Epistemic coprocessors

An epistemic interrupt means: “this emerging factual commitment requires external evidence.” A phrase such as “the product with the most sales in 2026 was ...” can trigger a database lookup before the model commits to a value. A phrase such as “X posted last week ...” can compose temporal resolution with web retrieval.

3 Generation-Time Assistance

Upfront RAG follows

$$Q \rightarrow R(Q) \rightarrow C \rightarrow \text{generate.}$$

Generation-time retrieval follows

$$Q \rightarrow y_{1:t} \rightarrow \text{epistemic interrupt} \rightarrow R(h_t) \rightarrow \text{evidence} \rightarrow y_{t+1:}.$$

The developing reasoning trajectory can expose needs that were not obvious from the original question.

4 Semantic Interrupt Ladder

1. strict syntax/grammar;
2. lexical and syntactic heuristics;
3. heuristic categorization/routing;
4. learned text-level detection and normalization;
5. hidden-state routing;
6. jointly trained neural-coprocessor interfaces.

5 Typed Micro-IR and State

Natural language is normalized into small typed events such as

$$\text{ADD}(10, 3), \quad \text{ISA}(\text{penguin}, \text{bird}),$$

or

$$\text{RETRIEVE}(\text{source}=\text{DB}, \text{relation}=\text{max_sales}, \text{year}=2026).$$

Outputs become facts, results, evidence spans, derivations, contradictions, freshness metadata, provenance and confidence.

6 Program Hypotheses

- H1 Reliability: coprocessors reduce exact-computation errors and unsupported factual commitments.
- H2 Efficiency: specialized engines reduce token and compute waste relative to neural emulation or verbose explicit tool use.
- H3 Scaling: heterogeneous systems degrade more slowly with reasoning depth, state size, and factual freshness.
- H4 Small-model leverage: smaller Transformers benefit disproportionately because they need only expose semantic structure and use returned state.

7 Interleaved Research Program

- Paper 0: Position: computational + epistemic coprocessors.
- Paper 1: Reflex calculator.
- Paper 1.5: Reflex retrieval, single source.
- Paper 2: Multiple compute/reasoning engines.
- Paper 2.5: Multiple retrieval engines.
- Paper 3: Learned computational semantic interrupts.
- Paper 3.5: Learned epistemic interrupts and routing.
- Paper 4: Transactional provenance/backtracking.
- Paper 5: Structured/PRA/KV interfaces.
- Paper 6: Co-adaptation / learning to offload and acquire.
- Paper 7: Integrated heterogeneous cognitive runtime.

8 Guardrails

Do not position symbolic engines as Transformer replacements. Do not assume tighter coupling is always better. Keep explicit tools and upfront RAG as baselines/fallbacks. Do not require PRA before state size makes selective materialization an empirical problem.

9 Related Work

The program is adjacent to tool-using LMs, program-aided reasoning, RAG and self-reflective retrieval, neuro-symbolic reasoning, and interactive theorem proving. Its specific focus is automatic generation-time computational and epistemic interrupts, persistent typed micro-state, and progressive coupling between neural and specialized computation.

References

- [1] Schick et al. Toolformer. NeurIPS, 2023.
- [2] Lewis et al. Retrieval-Augmented Generation. NeurIPS, 2020.
- [3] Asai et al. Self-RAG. ICLR, 2024.
- [4] Yao et al. ReAct. ICLR, 2023.
- [5] Gao et al. PAL. ICML, 2023.
- [6] Trinh et al. Solving Olympiad Geometry without Human Demonstrations. Nature, 2024.
- [7] Yang et al. LeanDojo. NeurIPS, 2023.